

M13 / PROTECTED ANSWERS

M13 Module Revision Test Answers

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 4

M13 | DELTA LAKE + LAKEHOUSE 4

PROTECTED TEST ANSWERS / VALIDATION 4

1. Foundations 4

2. Files vs tables 4

3. Transaction state 4

4. ACID 4

5. DML 4

6. Schema 4

7. Time travel 5

8. MERGE 5

9. Ambiguity 5

10. SCD2 5

11. Medallion 5

12. Maintenance 5

13. Maintenance 5

14. Incident 5

15. Runtime honesty 5

Practical validation 6

M13 practical worked solutions 6

P1 - executable merge pattern 6

P2 - reconcile a changed/new/unchanged SCD2 load 7

P3 - prove more than row accounting 7

P4 - recover before physical cleanup 8

P5 - commit progress last and explain partial failure 8

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

M13 | DELTA LAKE + LAKEHOUSE

PROTECTED TEST ANSWERS / VALIDATION

Open only after saving your closed-book module test. Use the explanations to repair gaps, then close this file and complete a fresh retry.

PROTECTED MATERIAL - open only after saving your closed-book attempt. Course runtime baseline: Spark 4.2.0 + Delta Lake 4.4.0. Live Delta/PySpark execution remains learner-PC proof unless explicitly recorded.

1. Foundations

A lake is primarily flexible lake/object storage; a warehouse supplies managed analytical table semantics; a lakehouse adds reliable table/transaction metadata over lake storage. Choose based on guarantees and workload, not a maturity ladder.

2. Files vs tables

Removed/old/uncommitted files can still physically exist. A Delta reader uses the transaction log to construct the committed snapshot; a naive Parquet scan can include inactive files.

3. Transaction state

Inspect the Delta transaction log/history/current version and reconcile the table snapshot. File creation alone does not prove commit.

4. ACID

Atomicity: do not expose a partial multi-file table change as the new table state. Isolation: concurrent operations reason about stable snapshots/conflicts rather than readers observing a half-written mixture.

5. DML

Preview predicate population in a sandbox, record before row/key/business totals, execute targeted delete, reconcile exact expected keys/count/totals, preserve history and recovery option before maintenance.

6. Schema

Reject unknown/unapproved breaking drift; quarantine when the batch cannot meet the curated contract but source evidence should be preserved; evolve only after compatibility, null/backfill and downstream impact are reviewed.

7. Time travel

Time travel reads an older snapshot; RESTORE creates a new commit making an old state current; VACUUM physically removes eligible unreferenced files and can make old snapshots unreadable.

8. MERGE

Stable business key, deterministic one-source-candidate-per-key, explicit precedence for stale/new changes, replayable source boundary, safe target commit, reconciliation and delayed external checkpoint/progress commit.

9. Ambiguity

Surface ambiguity and stop before target write. Do not let ROW_NUMBER/order or filesystem order choose an arbitrary winner when the business precedence is undefined.

10. SCD2

Exactly one current row per business key, historical validity intervals do not overlap, and a changed key expires one old current row while inserting one new current row; also preserve deterministic source ordering/change detection.

11. Medallion

Bronze_count = Silver_valid_count + quarantine_count for accounted scope; sum(Silver metric) = sum(Gold metric) for the same definition/scope. Add key controls as needed.

12. Maintenance

OPTIMIZE changes file layout/bin-packing, not business grain or deduplication logic. Duplicate rows remain duplicate rows unless transformation rules remove/reconcile them.

13. Maintenance

VACUUM can delete old physical files required for time travel/recovery and can interfere with long-running/lagging readers if retention is unsafe. It is not a harmless cleanup button.

14. Incident

Gold-to-Silver metric reconciliation -> Silver key/grain/quality controls -> Silver valid + quarantine vs Bronze -> Bronze vs source manifest -> table history/run metadata at first divergent boundary.

15. Runtime honesty

Static/simulator QA proves syntax/logic scaffolding and expected reasoning; learner-PC runtime proof shows the pinned Spark/Delta environment actually executed reads/writes/history/DML and produced saved evidence. Never substitute one for the other.

Practical validation

P1: precheck `groupBy(order_id).count > 1` -> fail; MERGE on stable key; matched update only when `source.updated_at > target.updated_at`; not matched insert; rerun same batch and prove row/key/total controls unchanged.

P2: before load prove one current target row/key; classify incoming new/changed/unchanged deterministically; expire only changed current rows; insert new versions/new keys; validate one current row/key, historical counts and non-overlap; rerun same batch for idempotency.

P3: $120 = 115 + 5$ proves row accounting only. Also prove Silver key uniqueness/grain, quarantine reasons, Gold revenue = Silver valid revenue for same scope, and source/Gold date/filter scope alignment.

P4: first preserve/investigate history and read pre-delete snapshot. Recover with a corrected forward write or RESTORE as appropriate and reconcile. OPTIMIZE can wait; VACUUM must wait because it can remove recovery files.

P5: durable source evidence/batch ID -> Bronze commit -> validate/reconcile -> Silver/quarantine commit -> stage deterministic source -> curated MERGE/SCD commit -> Gold build -> cross-layer controls -> save versions/evidence -> only then commit external checkpoint/progress.

M13 practical worked solutions

P1 - executable merge pattern

Run this inside the configured M13 Spark/Delta environment. `target_path` must name an existing disposable Delta table with `order_id`, `amount` and `updated_at`. source must have the same business columns and typed timestamps. The precheck deliberately rejects all repeated source keys in this practical; it does not choose an arbitrary winner. Validate fields and types before calling the function.

```
from delta.tables import DeltaTable
from pyspark.sql import functions as F

def apply_order_batch(spark, target_path, source):
    if source.filter(
        F.col("order_id").isNull()
        | F.col("updated_at").isNull()
        | F.col("amount").isNull()
    ).limit(1).count():
        raise ValueError("missing required source value")
    if source.groupBy("order_id").count().filter(
        "count > 1"
    ).limit(1).count():
        raise ValueError("ambiguous source key")
    target = spark.read.format("delta").load(target_path)
    if target.groupBy("order_id").count().filter(
        "count > 1"
    ).limit(1).count():
        raise ValueError("target grain already broken")
    same_version = source.alias("s").join(
        target.alias("t"),
        (F.col("s.order_id") == F.col("t.order_id"))
        & (F.col("s.updated_at") == F.col("t.updated_at"))
    )
```

```

if same_version.filter(
    ~F.col("s.amount").eqNullSafe(F.col("t.amount"))
).limit(1).count():
    raise ValueError("equal-version amount conflict")
table = DeltaTable.forPath(spark, target_path)
(
    table.alias("t")
    .merge(source.alias("s"), "t.order_id = s.order_id")
    .whenMatchedUpdate(
        condition="s.updated_at > t.updated_at",
        set={"amount": "s.amount",
            "updated_at": "s.updated_at"}
    )
    .whenNotMatchedInsertAll()
    .execute()
)

```

The matched predicate blocks stale updates. Equal-version identical rows leave the target unchanged, and an equal-version amount disagreement fails before the merge. In the supplied 06_merge_idempotency.py example, O1=50 and O2=80 start at total 130. O2 becomes 85 and O3=30 is inserted, so the final result has 3 rows/3 keys/165. Applying that batch again leaves the same controls; a stale O2=1 must not replace 85. The function assumes one writer during its precheck and merge. Production concurrency requires a transaction-aware design and conflict handling, not a promise that separate prechecks lock the table.

P2 - reconcile a changed/new/unchanged SCD2 load

Suppose the target begins with four customers and one current row per customer. The incoming batch changes two of them, leaves one unchanged and introduces one new customer. Expire two existing current rows, append two new versions for changed customers, and append one new customer. History grows from 4 to 7 rows, while current rows grow from 4 to 5. The unchanged customer gets no new version; the existing customer absent from the batch remains current unless the source contract says absence is deletion. Use half-open validity intervals [valid_from, valid_to). For each changed key, the old valid_to equals the new valid_from, so the change instant matches exactly one row. Test exactly one current row per active key, no overlapping history intervals, deterministic handling of duplicate/equal versions and an as-of join that preserves the fact row count. On exact replay, there are still 7 history rows and 5 current customers. These numbers demonstrate the logic; calculate the corresponding actual controls for your own fixture.

P3 - prove more than row accounting

120 = 115 + 5 explains all delivered rows only if the accepted and rejected sets are disjoint and cover the same batch. Also verify 115 valid keys at the intended Silver grain or explicitly account for a deduplication category. Preserve the five raw rejected rows and reasons. Sum the accepted Silver revenue using the same currency, status and date scope as Gold; it must independently equal 91250. Check relationships and explain any intentional exclusions. The number 91250 appearing in a dashboard is not a control unless it reconciles to the contributing rows.

P4 - recover before physical cleanup

First preserve history and identify the version immediately before the bad delete. Read that snapshot and compare the deleted key set and business amount with current state. A historical read is diagnostic and does not change current data. Once the intended recovery is established, a supported RESTORE or a carefully validated forward correction can create a new correct current version. Reconcile it before serving. OPTIMIZE may improve physical layout later; it cannot undelete business records. VACUUM waits because removing old files could destroy the recovery path you need.

P5 - commit progress last and explain partial failure

Capture source identity and raw bytes, validate and separate rejects, publish trusted current records, build Gold, reconcile across layers, save output versions and only then advance the external checkpoint. A table commit is not a transaction across all those steps. If Silver commits and Gold fails, consumers must keep using the last fully validated publication or see a clear unavailable state. Replay the source safely to rebuild Gold and commit progress. A single success badge cannot establish that every layer belongs to the same logical run.